

Question 1

Overview

This program simulates a simple binary classifier using McCulloch-Pitts neurons, functioning like a finite-state automaton in a 2D space. It defines three linear decision boundaries and combines their outputs to decide whether a point is accepted (green) or rejected (red).

Code

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def heavyside(x):
5      return np.where(x >= 0, 1, 0)
6
7  def sigmoid(x):
8      return 1 / (1 + np.exp(-x))
9
10 class McCulloch_Pitts_neuron:
11     def __init__(self, weights, bias, af):
12         self.weights = weights
13         self.bias = bias
14         self.af = af
15
16     def model(self, x):
17         return self.af(self.weights @ x + self.bias)
18
19 def DFA(input, af=heavyside, return_hidden=False):
20     lAB = McCulloch_Pitts_neuron(np.array([-2, -1]), 6, af)
21     lAC = McCulloch_Pitts_neuron(np.array([2, -1]), -2, af)
22     lBC = McCulloch_Pitts_neuron(np.array([0, 1]), 0, af)
23     neuron_and = McCulloch_Pitts_neuron(np.array([1, 1, 1]),
24                                         -2.5 if af==heavyside else -2, af)
25
26     z1 = lAB.model(input)
27     z2 = lAC.model(input)
28     z3 = lBC.model(input)
29     y = neuron_and.model([z1, z2, z3])
30
31     if return_hidden:
32         return y, [z1, z2, z3]
33     else:
34         return y
35
36 X = np.array([np.random.uniform(0,4,2000), np.random.uniform(-1,3,2000)])
37 y = DFA(X)
38 c = np.where(y, 'green', 'red')
```

```

39
40 plt.scatter(X[0], X[1], c=c)
41 plt.plot([1,2], [0,2], c='b')
42 plt.plot([2,3], [2,0], c='b')
43 plt.plot([1,3], [0,0], c='b')
44 plt.grid(True)
45 plt.show()

```

Explanation

- `McCulloch-Pitts_neuron` is a basic model neuron that returns either 0 or 1.
- Three boundary neurons (`1AB`, `1AC`, `1BC`) encode linear classifiers.
- The final neuron (`neuron_and`) combines them using a weighted sum and bias, mimicking a logical AND.
- The plot shows green for points classified as **accepted**, and red for **rejected**.
- Blue lines visualize the decision boundaries in the 2D plane.

Geometric Interpretation

The neurons define half-planes in 2D space. The intersection of the three half-planes defines a region (a triangle) where all neurons fire. The output neuron (AND logic) activates only if all three regions include the point.

1 Detailed Explanation of the McCulloch-Pitts DFA Classifier

The following classifier uses a simple neural model to simulate a geometric DFA (Deterministic Finite Automaton) in two dimensions. This is achieved using **McCulloch-Pitts neurons** — one of the earliest formal models of a biological neuron — which compute a weighted sum of inputs and apply a thresholded activation function.

Motivation for Using McCulloch-Pitts Neurons

McCulloch-Pitts neurons are ideal for modeling logical rules because:

- They produce binary outputs (0 or 1), similar to logical conditions.
- Their architecture is simple and interpretable.
- They are historically significant and form the foundation of neural computation.

Each neuron implements the rule:

$$y = f(\mathbf{w} \cdot \mathbf{x} + b)$$

where:

- $\mathbf{w}$ is the weight vector,
- $\mathbf{x}$ is the input vector,
- b is the bias (or threshold),
- f is the activation function (either Heaviside or Sigmoid).

Logical Structure of the Classifier

The classifier defines three intermediate neurons corresponding to three lines (decision boundaries):

- 1AB: separates region A and B.
- 1AC: separates region A and C.
- 1BC: separates region B and C.

Then, a final neuron called **neuron_and** combines the outputs of these three using logical AND-like behavior (i.e., it only activates when all three sub-neurons activate).

Step-by-Step Flow

1. Random Input Generation:

`np.random.uniform(0, 4, 2000)` generates 2000 random values for the X-axis, and similarly for the Y-axis.

2. Neuron Activation:

Each of the three neurons computes a linear combination of inputs and applies the activation function:

$$z_1 = \text{1AB.model}(\mathbf{x}), \quad z_2 = \text{1AC.model}(\mathbf{x}), \quad z_3 = \text{1BC.model}(\mathbf{x})$$

3. Final Classification:

The final neuron combines the hidden activations:

$$y = \text{neuron_and.model}([z_1, z_2, z_3])$$

It uses weights $[1, 1, 1]$ and a bias of -2.5 (in case of Heaviside), so the output is 1 only if all three hidden neurons output 1.

4. Color Assignment:

Green for accepted points ($y = 1$), Red for rejected points ($y = 0$).

5. Plotting:

The plot includes:

- A scatter plot of the classified data points.
- Three decision boundary lines (manually drawn for visual clarity).

Conclusion

This program simulates a rudimentary finite automaton using interpretable neural logic gates. By designing geometric regions via linear decision boundaries, and applying logic via McCulloch-Pitts neurons, it serves as a powerful didactic example of how neural systems can model rule-based classifiers.

2 Using Sigmoid Activation for Soft Classification

In this variation, the activation function used by the McCulloch-Pitts neurons is changed from a binary **heaviside** function to a smooth **sigmoid** function. This transforms the system from a discrete logic gate simulator into a **soft classifier** that models probabilities.

Code Differences and Explanation

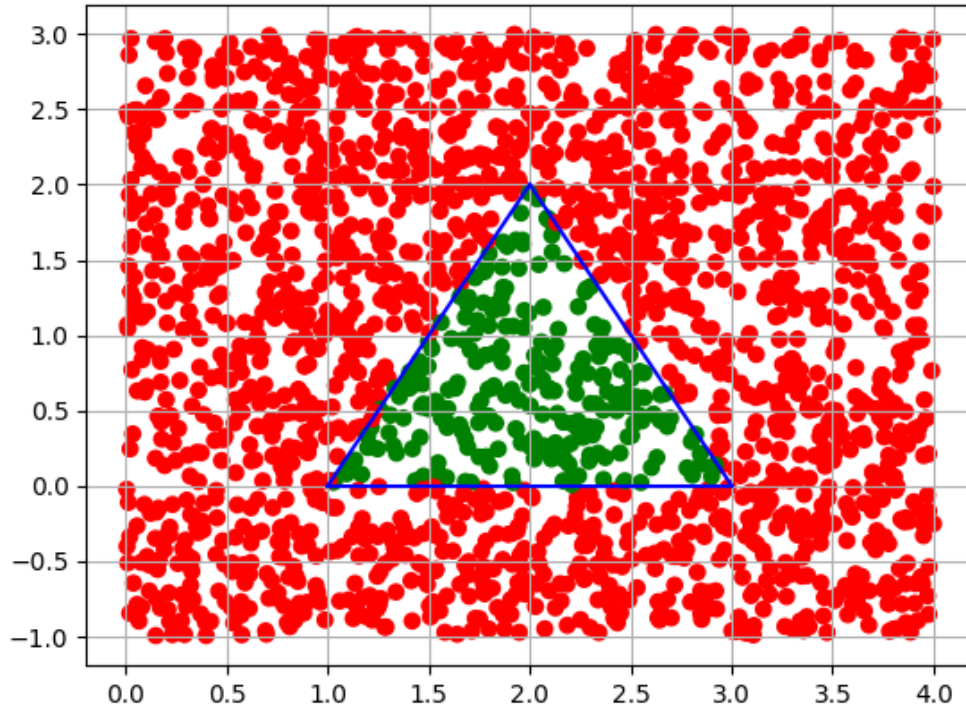


Figure 1: Visualization of classified 2D points using the McCulloch-Pitts neuron-based DFA. Green points are accepted, red points are rejected. Blue lines represent decision boundaries.

```

1 y = DFA(X, sigmoid)
2 c = np.where(y >= 0.5, 'green', 'red')
3 plt.scatter(X[0], X[1], c=c)
4 plt.plot([1, 2], [0, 2], c='b')
5 plt.plot([2, 3], [2, 0], c='b')
6 plt.plot([1, 3], [0, 0], c='b')
7 plt.grid(True)
8 plt.show()

```

What Changed?

- The DFA model is now called with `sigmoid` as the activation function.
- Each neuron now returns a value in the range $[0, 1]$ instead of just 0 or 1.
- To turn these probabilities into binary decisions, we apply a threshold:

`np.where( $y \geq 0.5$ , 'green', 'red')`

- This maintains consistency with the previous visualization (green for accepted, red for rejected), but now reflects a smoother decision surface.

Why Sigmoid?

Using the sigmoid activation has the following effects:

1. **Continuity:** The model can handle small variations in input smoothly.

2. **Probabilistic output:** The final layer's output can be interpreted as confidence in the classification.
3. **Better visualization:** Especially useful when visualizing boundaries in a continuous space.

Visual Output (Image 2)

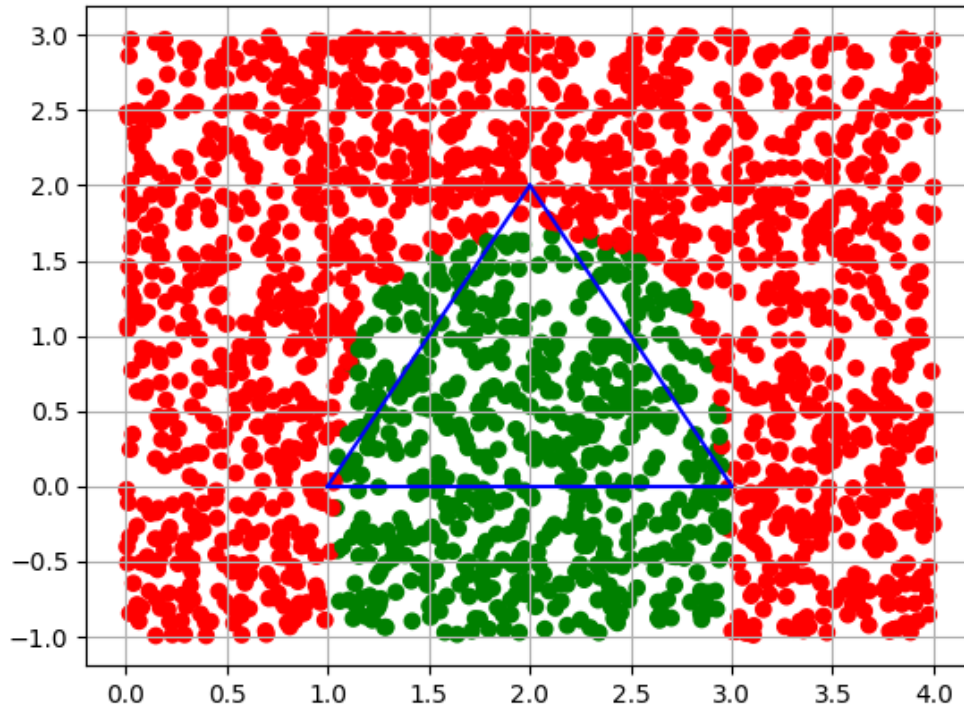


Figure 2: Soft classification using McCulloch-Pitts neurons with sigmoid activation. The boundary becomes smoother compared to the sharp Heaviside classifier.